

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Dataset Integration

COURSE NAME: Query Processing for
Data Science

COURSE CODE: DSA0503

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 30%

Total Marks: 20

Code of Conduct

I **UDHAYA KUMAR U-192424332** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Dataset Integration Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Dataset Understanding & Preparation	30%	Correctly understands both datasets, identifies relevant columns/keys, and loads them properly.	Minor misunderstanding or preparation issue.	Loads datasets but needs some guidance in identifying relevant fields.	Unable to identify or prepare the datasets correctly.
Dataset Integration	20%	Correctly integrates datasets using appropriate merge(), join(), or concat() and selects the correct integration key.	Integration is mostly correct with minor errors.	Performs integration with significant guidance or minor key/operation errors.	Unable to integrate the datasets correctly.
Data Processing & Analysis	15%	Correctly performs all required calculations, filtering, grouping, or transformations on the integrated data.	Performs most required analysis correctly.	Performs basic analysis but misses some required operations.	Analysis is incorrect or incomplete.
Python/Pandas	20%	Code is correct,	Code works with	Code works	Code contains major

Implementation		efficient, well-structured, and uses appropriate Pandas functions.	minor inefficiencies/errors.	partially and requires corrections.	errors or does not execute.
Output & Interpretation	15%	Clearly presents the required output and correctly interprets the results.	Output is correct with limited interpretation.	Output is partially correct or poorly presented.	Output is missing/incorrect and results are not interpreted.

Total Marks Awarded:

Signature of the Course Faculty

Questions:

Set 1 – Dataset Integration Tasks

1. Employee–Department Integration

You are given two datasets:

- employees.csv — Employee_ID, Employee_Name, Department_ID, Salary
- departments.csv — Department_ID, Department_Name, Location

Task: Write a Python program using Pandas to integrate the two datasets based on Department_ID and create a new dataset containing employee name, department name, location, and salary.

Code:

```
import pandas as pd
```

```
# Create employee dataset
```

```
employees = pd.DataFrame({  
    "Employee_ID": [101, 102, 103, 104],  
    "Employee_Name": ["Arun", "Bala", "Charan", "Divya"],  
    "Department_ID": [1, 2, 1, 3],  
    "Salary": [30000, 40000, 35000, 45000]  
})
```

```
# Create department dataset
```

```
departments = pd.DataFrame({  
    "Department_ID": [1, 2, 3],  
    "Department_Name": ["IT", "HR", "Finance"],  
    "Location": ["Chennai", "Bangalore", "Mumbai"]  
})
```

```
# Save as CSV files
```

```
employees.to_csv("employees.csv", index=False)
```

```

departments.to_csv("departments.csv", index=False)

# Integrate datasets

result = pd.merge(employees, departments, on="Department_ID")

# Select required columns

result = result[["Employee_Name", "Department_Name", "Location", "Salary"]]

print("Employee-Department Integrated Dataset:")

print(result)

```

Output:

```

Employee-Department Integrated Dataset:
  Employee_Name  Department_Name  Location  Salary
0         Arun                IT    Chennai   30000
1        Charan                IT    Chennai   35000
2         Bala                HR   Bangalore   40000
3        Divya               Finance    Mumbai   45000

```

2. Customer–Transaction Integration

You are given:

- customers.csv — Customer_ID, Customer_Name, City
- transactions.csv — Transaction_ID, Customer_ID, Product, Amount

Task: Integrate the datasets using Customer_ID. Display the customer name, city, product, and transaction amount. Identify customers who have made transactions above ₹10,000.

Code:

```

import pandas as pd

# Create customer dataset

customers = pd.DataFrame({

    "Customer_ID": [1, 2, 3, 4],

    "Customer_Name": ["Ravi", "Priya", "Kumar", "Anu"],

```

```
"City": ["Chennai", "Madurai", "Coimbatore", "Salem"]
})
```

```
# Create transaction dataset
```

```
transactions = pd.DataFrame({
    "Transaction_ID": [101, 102, 103, 104],
    "Customer_ID": [1, 2, 3, 4],
    "Product": ["Laptop", "Mobile", "Tablet", "Laptop"],
    "Amount": [55000, 8000, 12000, 45000]
})
```

```
# Save as CSV
```

```
customers.to_csv("customers.csv", index=False)
transactions.to_csv("transactions.csv", index=False)
```

```
# Integrate datasets
```

```
result = pd.merge(customers, transactions, on="Customer_ID")
```

```
# Display required columns
```

```
display_data = result[["Customer_Name", "City", "Product", "Amount"]]
```

```
print("Customer-Transaction Integrated Dataset:")
```

```
print(display_data)
```

```
# Customers with transactions above 10000
```

```
print("\nCustomers with transactions above Rs.10,000:")
```

```
print(display_data[display_data["Amount"] > 10000])
```

Output:

Customer-Transaction Integrated Dataset:

	Customer_Name	City	Product	Amount
0	Ravi	Chennai	Laptop	55000
1	Priya	Madurai	Mobile	8000
2	Kumar	Coimbatore	Tablet	12000
3	Anu	Salem	Laptop	45000

Customers with transactions above Rs.10,000:

	Customer_Name	City	Product	Amount
0	Ravi	Chennai	Laptop	55000
2	Kumar	Coimbatore	Tablet	12000
3	Anu	Salem	Laptop	45000

3. Student-Marks Integration

You are given:

- students.csv — Student_ID, Student_Name, Department
- marks.csv — Student_ID, Python, DBMS, Statistics

Task: Merge the datasets using Student_ID. Create a new column Total_Marks and identify students whose total marks are greater than 240.

Code:

```
import pandas as pd
```

```
# Create student dataset
```

```
students = pd.DataFrame({  
    "Student_ID": [101, 102, 103, 104],  
    "Student_Name": ["Arun", "Bala", "Charan", "Divya"],  
    "Department": ["AI&DS", "CSE", "AI&DS", "ECE"]  
})
```

```
# Create marks dataset
```

```
marks = pd.DataFrame({  
    "Student_ID": [101, 102, 103, 104],  
    "Python": [85, 90, 75, 95],  
    "DBMS": [80, 85, 90, 80],  
    "Statistics": [90, 75, 85, 95]  
})
```

```
# Save as CSV
```

```
students.to_csv("students.csv", index=False)  
marks.to_csv("marks.csv", index=False)
```

```
# Merge datasets
result = pd.merge(students, marks, on="Student_ID")
```

```
# Calculate total marks
result["Total_Marks"] = (
    result["Python"] +
    result["DBMS"] +
    result["Statistics"]
)
```

```
print("Student-Marks Integrated Dataset:")
print(result)
```

```
# Students with total marks greater than 240
print("\nStudents with Total Marks greater than 240:")
print(result[result["Total_Marks"] > 240])
```

Output:

```
Student-Marks Integrated Dataset:
   Student_ID Student_Name Department  Python  DBMS  Statistics  Total_Marks
0          101         Arun    AI&DS      85    80          90          255
1          102         Bala      CSE      90    85          75          250
2          103        Charan    AI&DS      75    90          85          250
3          104        Divya      ECE      95    80          95          270

Students with Total Marks greater than 240:
   Student_ID Student_Name Department  Python  DBMS  Statistics  Total_Marks
0          101         Arun    AI&DS      85    80          90          255
1          102         Bala      CSE      90    85          75          250
2          103        Charan    AI&DS      75    90          85          250
3          104        Divya      ECE      95    80          95          270
```

4. Product-Sales Integration

You are given:

- products.csv — Product_ID, Product_Name, Category, Unit_Price
- sales.csv — Sale_ID, Product_ID, Quantity, Sale_Date

Task: Integrate the two datasets using Product_ID. Calculate the total sales amount for each transaction using:

$\text{Total_Sales} = \text{Quantity} \times \text{Unit_Price}$

Display the integrated dataset.

Code:

```
import pandas as pd

# Create product dataset

products = pd.DataFrame({

    "Product_ID": [1, 2, 3, 4],

    "Product_Name": ["Laptop", "Mouse",
"Keyboard", "Monitor"],

    "Category": ["Electronics",
"Accessories", "Accessories",
"Electronics"],

    "Unit_Price": [50000, 800, 1500, 12000]

})

# Create sales dataset

sales = pd.DataFrame({

    "Sale_ID": [101, 102, 103, 104],

    "Product_ID": [1, 2, 3, 4],

    "Quantity": [2, 5, 3, 2],

    "Sale_Date": ["2026-08-01", "2026-08-
02", "2026-08-03", "2026-08-04"]

})

# Save as CSV

products.to_csv("products.csv",
index=False)

sales.to_csv("sales.csv", index=False)
```



```
# Integrate datasets

result = pd.merge(sales, products,
on="Product_ID")

# Calculate total sales

result["Total_Sales"] = result["Quantity"] *
result["Unit_Price"]

print("Product-Sales Integrated Dataset:")

print(result)
```

Output:

```
Product-Sales Integrated Dataset:
   Sale_ID  Product_ID  Quantity  ...  Category  Unit_Price  Total_Sales
0      101           1         2  ...  Electronics      50000      100000
1      102           2         5  ...  Accessories       800         4000
2      103           3         3  ...  Accessories      1500         4500
3      104           4         2  ...  Electronics     12000      24000

[4 rows x 8 columns]
```

5. Hospital Patient–Visit Integration

You are given:

- patients.csv — Patient_ID, Patient_Name, Age, Gender
- visits.csv — Visit_ID, Patient_ID, Doctor, Visit_Date, Fee

Task: Integrate the patient and visit datasets using Patient_ID. Display patient name, age, doctor, visit date, and fee. Calculate the total consultation fee paid by each patient.

Code:

```
import pandas as pd

# Create patient dataset

patients = pd.DataFrame({
    "Patient_ID": [1, 2, 3, 4],
    "Patient_Name": ["Ravi", "Priya", "Kumar", "Anu"],
```

```

    "Age": [25, 30, 45, 35],
    "Gender": ["Male", "Female", "Male", "Female"]
})

# Create visit dataset
visits = pd.DataFrame({
    "Visit_ID": [101, 102, 103, 104, 105],
    "Patient_ID": [1, 2, 1, 3, 4],
    "Doctor": ["Dr. Kumar", "Dr. Priya", "Dr. Ravi", "Dr. Arun", "Dr. Meena"],
    "Visit_Date": ["2026-08-01", "2026-08-02", "2026-08-03", "2026-08-04", "2026-08-05"],
    "Fee": [500, 700, 600, 800, 500]
})

# Save as CSV
patients.to_csv("patients.csv", index=False)
visits.to_csv("visits.csv", index=False)

# Integrate datasets
result = pd.merge(patients, visits, on="Patient_ID")

# Display required information
display_data = result[
    ["Patient_Name", "Age", "Doctor", "Visit_Date", "Fee"]
]

print("Patient-Visit Integrated Dataset:")
print(display_data)

```

```
# Calculate total consultation fee for each patient
```

```
total_fee = result.groupby(  
    ["Patient_ID", "Patient_Name"]  
)["Fee"].sum()
```

```
print("\nTotal Consultation Fee Paid by Each Patient:")
```

```
print(total_fee)
```

Output:

```
Patient-Visit Integrated Dataset:
```

	Patient_Name	Age	Doctor	Visit_Date	Fee
0	Ravi	25	Dr. Kumar	2026-08-01	500
1	Ravi	25	Dr. Ravi	2026-08-03	600
2	Priya	30	Dr. Priya	2026-08-02	700
3	Kumar	45	Dr. Arun	2026-08-04	800
4	Anu	35	Dr. Meena	2026-08-05	500

```
Total Consultation Fee Paid by Each Patient:
```

Patient_ID	Patient_Name	
1	Ravi	1100
2	Priya	700
3	Kumar	800
4	Anu	500

```
Name: Fee, dtype: int64
```